

ASTANA IT UNIVERSITY

Smart Content Recommendation System

Final Project | Introduction to Programming 2 (Python)

Dinara

Lead / Architect

Marzhan

Data Specialist

Balzhan

Logic Developer

Aigerim

QA / Integration

1. PROBLEM & PROPOSED SOLUTION

The Challenge

Choice overload is a major issue in streaming. Users spend more time searching than watching. Static, hard-coded libraries lack the flexibility to scale with user growth.

Our Vision

A dynamic, disk-persistent engine that calculates "Social Similarity." It leverages shared tastes to bridge the gap between discovery and consumption.

Mathematical Engine

We implemented **Set Theory** for high-speed analysis. By identifying overlapping interests between users, the engine predicts the "next best watch."



"Finding content you'll love through people like you."

2. SYSTEM DESIGN: OOP ARCHITECTURE

Content

Abstract base class for media items. Handles universal IDs and categorization.

Movie

Concrete implementation with specialized attributes like runtime and studio metadata.

User

Profile management class using set() for high-performance history tracking.

Engine

The "Brain" of the system. Orchestrates JSON I/O and similarity matching logic.

Syllabus Compliance:

- ✓ **Encapsulation:** Private data states via @property.
- ✓ **Inheritance:** Shared logic through super().__init__().
- ✓ **Polymorphism:** Method overriding for dynamic profiles.
- ✓ **Abstraction:** Separation of data and logic layers.

3. DECISIONS: ALGORITHMIC OPTIMIZATION

$O(1)$

Average Lookup
Complexity

Replacing nested for loops ($O(n^2)$) with Python Hash
Sets.

Performance Breakthrough

Initial designs for similarity matching were slow. By utilizing **Set Intersections and Differences**, we moved processing from the Python interpreter to efficient C-level operations.

```
# High-speed matching logic intersection =  
u1.history & u2.history recs = list(u2.history -  
u1.history)
```

Result: Instant recommendations even with thousands of user records.

4. ADVANCED IMPLEMENTATION FEATURES

Streaming Generators

We integrated **Generators** using the `yield` keyword. This ensures the system only processes one movie at a time during large library scans, keeping RAM usage extremely low.

Functional Filtering

Leveraged lambda functions with `filter()` for dynamic catalog queries. This allows users to filter recommendations by genre or year without complex nested logic.

 **Robust Error Handling:** We implemented a global try-except architecture to handle corrupted JSON data, missing files, and invalid user input without crashing the main simulation.

5. DATA PERSISTENCE STRATEGY

The system maintains a persistent state by utilizing structural **JSON** serialization. This allows for seamless "Save & Resume" functionality.

- ✓ users.json: Stores history matrix and IDs.
- ✓ items.json: Static content database.

Dynamic Disk Write

When a user updates their watch history during a session, the system automatically triggers a **Disk Commit**. Changes are saved instantly to the physical file, preventing data loss.



6. PROGRAM DEMO & VALIDATION

Interactive Console UI

Our main.py script simulates a production environment with login capabilities, profile analytics, and real-time recommendation feeds.

```
$ python main.py [Success] Database loaded. Enter User ID: 1  
Recommendations for Dinara: - Interstellar (ID: 101) -  
Inception (ID: 105)
```

Automated Testing

We developed test_project.py using strict assert statements to verify OOP integrity and algorithm precision.

```
$ python test_project.py .....  
_____ RAN 25 TESTS: OK
```

7. TEAM CONTRIBUTION BREAKDOWN

Member	Role	Key Contributions
Dinara	Lead / Architect	Class Hierarchy, Encapsulation, Repository Workflow.
Marzhan	Data Specialist	JSON Persistence, File I/O operations, Data Matrix.
Balzhan	Logic Developer	Set-theoretic logic, Generators, Lambda filters.
Aigerim	QA / Integration	Control Flow, Error Handling, Unit Testing Suite.

Thank you! Questions?